

rawk

rawk.fastrp

fastrp_merge(U_list, weights, normalization=False)

[Function]

fastrp_projection(

A,

q=3,

dim=128,

projection_method='gaussian',

input_matrix='adj',

alpha=None) [Function]

fastrp_wrapper(A, conf) [Function]

rawk.input_prep

**get_met_net_dfs(
 rxn_gene_df,
 rxn_edge_df,
 gene_prop_df,
 mn_weight_cutoff,
 fill_missing_gene_prop=0,
 transform_gene_prop_func=None,
 rxn_gene_prop_agg_func=None) [Function]**

Prepare input node and edge dataframes for Rawk

Parameters

rxn_gene_df : dataframe

 A dataframe of reactions and their associated genes.

Following are the

 required columns: 'rxn' (reaction ID),

 'rxn_name' (reaction name),

 'equation' (reaction equation), 'pathway' (reaction
pathway), and

 'gene' (gene symbol). If a reaction is associated with
multiple genes,

 one row lists one associated gene.

rxn_edge_df : dataframe

 A dataframe of reactions edges. Following are the
required columns:

 'src' (source node reaction ID), 'dest' (destination node
reaction ID),

 'mn_weight' (metabolic network edge weight). The edges
are undirected,

 with the src <= dest in alphabetical order.

gene_prop_df : dataframe

 A dataframe of gene properties, such as log fold changes,
normalized

 read counts, and z-scores. The 'gene' column of the
dataframe is a

```

    list of gene symbols. Other columns of the dataframe are
the gene
    properties of samples.
mn_weight_cutoff : float
    The cutoff of metabolic network weights. Keep only edges
with weights >
    mn_weight_cutoff.
fill_missing_gene_prop : int, or float, or None
    If int or float, replace missing gene properties with this
value.
    If None, drop genes with missing properties.
transform_gene_prop_func : function
    A function to transform each property column.
rxn_gene_prop_agg_func : function or None
    The function used to aggregate the property values of
multiple genes
    that are associated with each reaction. If None, use
    lambda x: x.mean() to aggregate.

Returns
-----
(dataframe, dataframe)
    A 2-tuple of a reaction node property dataframe and a
reaction edge
    dataframe

```

**get_mrn_gp_df(
 gene_prop_df,
 rxn_gene_df,
 fill_missing_gene_prop=None,
 transform_gene_prop_func=None) [Function]**

Prepare metabolic reaction network gene property dataframe

Parameters

gene_prop_df : dataframe

A dataframe of gene properties, such as log fold changes,
normalized

read counts, and z-scores. The 'gene' column of the dataframe is a list of gene symbols. Other columns of the dataframe are the gene properties of samples.

`rxn_gene_df` : dataframe

A dataframe of reactions and their associated genes. Following are the required columns: 'rxn' (reaction ID), 'rxn_name' (reaction name), 'equation' (reaction equation), 'pathway' (reaction pathway), and 'gene' (gene symbol). If a reaction is associated with multiple genes, one row lists one associated gene.

`fill_missing_gene_prop` : int, or float, or None

If int or float, replace missing gene properties with this value.

If None, drop genes with missing properties.

`transform_gene_prop_func` : function

A function to transform each property column. If None, no transformation will be applied.

Returns

dataframe

**qn_transform(
 s,
 sigma=0.367879,
 log1p=False,
 collapse_0s=False,
 center=False,
 seed=42) [Function]**

Quantile transform gene properties to normal distribution

Parameters

```

-----
s : series
    A pandas.Series of gene properties, such as log fold
changes,
    normalized read counts, and z-scores.
sigma : float
    Output normal distribution sigma. Default to
    np.round(1 / np.e, 6), so np.e ** (sigma * (3 - (-3))) ~=
9.
loglp : bool
    Apply loglp transform on the properties or not, before
    quantile normalization.
collapse_0s : bool
    If True, all 0s will be collapse into one 0, and the 0's
    quantile normalized value will be assigned to all 0s.
center : bool
    If True, the tranformed values will be centered at the
input 0s.
seed : int
    Random state.

Returns
-----
Series after transformation

```

transform_gene_prop(gene_prop_df, transform) [Function]

```

Apply transform function to each property column

Parameters
-----
gene_prop_df : dataframe
    A dataframe of gene properties, such as log fold changes,
normalized
    read counts, and z-scores. The 'gene' column of the
dataframe is a
    list of gene symbols. Other columns of the dataframe are
the gene
    properties of samples.
transform : function
    A function to transform each property column.

```

Returns

dataframe after transformation

rawk.multisample_rawk

MultiSampleRawk(

```
node_df,  
edge_df,  
bg_sample_col=None,  
n_jobs=1,  
seed=17,  
node_emb_method='fastrp',  
n2v_walk_length=20,  
n2v_num_walks=8000,  
n2v_chunk_num_walks=1000,  
n2v_p=1.0,  
n2v_q=1.0,  
n2v_dimensions=50,  
n2v_window=10,  
n2v_epochs=1,  
random_walk_n_steps_method=None,  
n_workers_per_sample=1,  
verbose=False) [Class]
```

Run Rawk on multiple samples

Parameters

node_df : dataframe

The reaction node dataframe with the following columns:

```

    'rxn', 'pathway', 'gene', 'sample1', 'sample2',
    'sample3', ....
    Each value of the 'gene' column is a ;;; joined list of
gene
    symbols.
edge_df : dataframe
    The reaction edge dataframe with the following columns:
    'src', 'dest', 'mn_weight'.
bg_sample_col : str or None
    The column in node_df used as background. If
bg_sample_col is None, a
    uniform background sample will be created.
n_jobs : int
    The number of jobs for running each sample parallely.
Note that
    the number of CPUs for running each sample is set by
    n_workers_per_sample.
seed : int
    Random number seed.
node_emb_method : str or dict
    The method to set node embeddings, which can be one of
the following
    options:
    - 'fastrp': Compute 2D t-SNE from fastrp node embeddings.
    - 'node2vec': Compute 2D t-SNE from node2vec node
embeddings.
    This procedure may take several hours.
    - dict: Reuse pre-computed 2D embeddings of nodes stored
in a dict
    with the following format:
    {node_name1: (dim1_coord, dim2_coord), ...}.
n2v_walk_length : int
    node2vec walk length parameter.
n2v_num_walks : int
    node2vec number of walks parameter.
n2v_chunk_num_walks : int
    node2vec number of walks per chunk. Run node2vec in
chunks to reduce
    mem usage.
n2v_p : float
    node2vec p parameter.
n2v_q : float
    node2vec q parameter.

```

```

n2v_dimensions : int
    node2vec number of representation dimensions.
n2v_window : int
    node2vec window parameter.
n2v_epochs : int
    node2vec number of epochs parameter.
random_walk_n_steps_method : str
    Generate random walk number of steps with one of the
    following methods:
    - 'simulation': Simulate random walks. Aggregate number
    of steps.
    - 'calculation': Calculate number of steps using
    transition probability
    matrix. NOTE: This method is only implemented for
    n2v_p=1 and n2v_q=1.
    If node_emb_method='node2vec', the random walks still
    need to be
    simulated.
    - None: When n2v_p=1 and n2v_q=1, use the calculation
    method. Otherwise,
    use the simulation method.
n_workers_per_sample : int
    The number of workers to run each sample.
verbose : bool
    Print runtime messages or not.

```

MultiSampleRawk.set_node_emb(node_emb_method, pca_ndim=None, kmeans_n_clust=None) [Method]

Set node embeddings

Parameters

node_emb_method : str or dict

The method to set node embeddings, which can be one of the following options:

- 'fastrp': Compute 2D t-SNE from fastrp node embeddings.
- 'node2vec': Compute 2D t-SNE from node2vec node

embeddings.

This procedure may take several hours.

```

    - dict: Reuse pre-computed 2D embeddings of nodes stored
in a dict
    with the following format:
    {node_name1: (dim1_coord, dim2_coord), ...}. In this
option,
    pca_ndim and kmeans_n_clust should be None.
pca_ndim : int or None
    When node_emb_method in ['fastrp', 'node2vec'], the
number of PCA
    dimensions of the fastrp or node2vec embeddings to
compute 2D t-SNE
    and kmeans clusters. If pca_ndim is None, default
pca_ndim=6.
kmeans_n_clust : int or None
    When node_emb_method in ['fastrp', 'node2vec'], the
number of
    clusters parameter to run kmeans clustering. If None,
default
    pca_ndim=10.

Returns
-----
None

```

**MultiSampleRawk.test_num_steps(h_exponent=1, cmp_norm_fac=5000,
pw_subset=None) [Method]**

```

Enrichment test using the number of random walk steps

Parameters
-----
h_exponent : float
    The heuristic factor for exponentiating sample vs
background
    fold changes of the number of random walk steps. This
heuristic
    factor intends to distinguish pathways with relatively
small and
    large effect sizes.
cmp_norm_fac : float

```

```

    Normalization factor for the number of random walk steps.
pw_subset : set, or list, or None
    The subset of pathways to keep in the result dataframes.
    If is None, keep all pathways with not-NA results.

Returns
-----
A 2-tuple of (fdr_dataframe, enrichment_score_dataframe).

```

**MultiSampleRawk.test_property_values(rw_s_prop_cutoff=0.005,
s_ppd_dict=None, pw_subset=None) [Method]**

```

Enrichment test using the node property values

Parameters
-----
rw_s_prop_cutoff : float
    The cutoff proportion of random walk steps used for
    defining local
    pathways.
s_ppd_dict : dict or None
    {sample1: {pathway1: [property_value1, ...], ...}, ...}
    dict used
    for extracting pathway property values for testing. If
    None, use
    the foreground sample node properties.
pw_subset : set, or list, or None
    The subset of pathways to keep in the result dataframes.
    If is None, keep all pathways with not-NA results.

Returns
-----
A 2-tuple of (fdr_dataframe, enrichment_score_dataframe).

```

MultiSampleRawkTest(rawk_test_list) [Class]

```

Run RawkTest on multiple samples

```

```

Parameters

```

```
-----  
rawk_test_list : list of RawkTest
```

MultiSampleRawkTest.from_list(dl) [Method]

Construct a MultiSampleRawkTest from a list

MultiSampleRawkTest.load_json(file_path) [Method]

```
Read json files  
  
Parameters  
-----  
file_path : str  
    Read file prefix.  
  
Returns  
-----  
A MultiSampleRawkTest constructed from the read data.
```

MultiSampleRawkTest.merge_test_res(test_res_list) [Method]

Merge a list of test results generated by
Rawk.test_res_dict_to_df

MultiSampleRawkTest.save_to_json(file_path) [Method]

```
Save data to a json file  
  
Parameters  
-----  
file_path : str  
    Save json file path.  
  
Returns
```

```
-----  
None
```

**MultiSampleRawkTest.test_num_steps(h_exponent=1,
cmp_norm_fac=5000, pw_subset=None) [Method]**

```
Enrichment test using the number of random walk steps  
  
Parameters  
-----  
h_exponent : float  
    The heuristic factor for exponentiating sample vs  
background  
    fold changes of the number of random walk steps. This  
heuristic  
    factor intends to distinguish pathways with relatively  
small and  
    large effect sizes.  
  
cmp_norm_fac : float  
    Normalization factor for the number of random walk steps.  
  
pw_subset : set, or list, or None  
    The subset of pathways to keep in the result dataframes.  
    If is None, keep all pathways with not-NA results.  
  
Returns  
-----  
A 2-tuple of (fdr_dataframe, enrichment_score_dataframe).
```

**MultiSampleRawkTest.test_property_values(rw_s_prop_cutoff=0.005,
s_ppd_dict=None, pw_subset=None) [Method]**

```
Enrichment test using the node property values  
  
Parameters  
-----  
rw_s_prop_cutoff : float
```

```

    The cutoff proportion of random walk steps used for
    defining local
    pathways.
s_ppd_dict : dict or None
    {sample1: {pathway1: [property_value1, ...], ...}, ...}
dict used
    for extracting pathway property values for testing. If
None, use
    the foreground sample node properties.

pw_subset : set, or list, or None
    The subset of pathways to keep in the result dataframes.
    If is None, keep all pathways with not-NA results.

Returns
-----
A 2-tuple of (fdr_dataframe, enrichment_score_dataframe).

```

MultiSampleRawkTest.to_list() [Method]

```

Convert data to a list

```

rawk.n2v

```
RawkNode2Vec(  
    node_df,  
    ea_df,  
    p=1.0,  
    q=1.0,  
    num_walks=8000,  
    chunk_num_walks=1000,  
    walk_length=20,  
    workers=1,  
    seed=17,  
    keep_walks=False) [Class]
```

Mem efficient chunked Node2Vec

```
RawkNode2Vec.get_visit_arr() [Method]
```

rawk.plot

**adjust_plot_pos(
 pos,
 node_sizes_in,
 fig,
 ax,
 padding_in=0.1,
 iterations=200,
 force_step_in=0.05,
 seed=42)** [Function]

assert_allclose(x, y) [Function]

assert_frame_equal(x, y) [Function]

expand_v0range(vmin, vmax) [Function]

If `vmin == vmax`, make them different

**get_color_mapper(vmin, vmax, vcenter=None,
cmap_name=None)** [Function]

Creates a color mapper function.

Parameters:

`vmin, vmax` : float

The minimum and maximum limits of the data.

`vcenter` : float, optional

The center point (e.g., 0). If None, standard linear mapping is used.

`cmap_name` : str, optional

The name of the Matplotlib colormap. If None, centered

```
use seismic, otherwise viridis.
```

Returns:

mapper : matplotlib.cm.ScalarMappable

An object with a `.to_rgba(value)` method to fetch colors.

```
hist(  
  data,  
  n_bins,  
  title,  
  xlab,  
  ylab,  
  filename,  
  figsize=(1.8,  
  1.5)) [Function]
```

```
map_strings_to_colors(strings, palette) [Function]
```

```
plot_elbow(metric_by_n, selected_n, save_pfx, xlab, ylab,  
annot) [Function]
```

```
plot_graph(  
    G,  
    pos,  
    filename,  
    s_pw=None,  
    node_color_attr='property',  
    draw_edges=False,  
    draw_labels=False,  
    non_s_pw_node_size=10,  
    s_pw_node_size=150,  
    title=None,  
    figsize=(3,  
    3),  
    seed=42,  
    dim_name='Dim',  
    discrete_color=False,  
    cmap=None,  
    node_alpha=1,  
    edge_width=None,  
    dpi=300,  
    discrete_col_legend_ncol=3,  
    edge_alpha=None) [Function]
```

Plot networkx graph

Parameters

cmap : str, dict, optional

If discrete_color, cmap is a dictionary to map node_color_attr values to colors. If not discrete_color, cmap is a string. If None, discrete_color cmap is from R package ggsci::pal_jco, and not discrete_color cmap is 'viridis'.

Returns

None

```
plot_mtx(  
    mtx,  
    row_labels,  
    col_labels,  
    cbar_title,  
    figsize,  
    filename,  
    cbar_hm_space,  
    vmin=None,  
    vmax=None) [Function]
```

```
plot_nw_stats(  
    uf_ntbl,  
    uf_etbl,  
    mn_weight_cutoff,  
    mn_weight_cutoff_label,  
    title_pfx,  
    fn_pfx) [Function]
```

```

plot_pw_neighborhood(
    rawk,
    s_pw,
    filename,
    node_color_attr,
    mean_node_color_attr=True,
    node_color_vmin=None,
    node_color_vmax=None,
    node_color_center=None,
    n_cutoff=10,
    node_alpha=0.5,
    node_color_title=None,
    title=None,
    figsize=None,
    draw_labels=True) [Function]

```

Plot pathway reachable local neighborhood

Parameters

rawk : Rawk

A Rawk instance.

s_pw : str

A pathway to plot.

filename : str

Saved plot filename

node_color_attr : str

Node addtribute used for coloring:

- 'property': node property
- 'rwv_cpm': random walk visit count per million
- 'log10p1_rwv_cpm': $\log_{10}(\text{rwv_cpm} + 1)$
- 'rwv_cpm_diff': sample - background rwv_cpm
- 'rwv_cpm_p1_log2_fc': $\log_2((\text{sample_rwv_cpm} + 1) / (\text{background_rwv_cpm} + 1))$

mean_node_color_attr : bool

If True, plot the mean node_color_attr of

```
    the reactions in each pathway node. If False,
    plot each pathway node as a pie chart, with each
    wedge as the node_color_attr of each reaction in
    the pathway.
node_color_vmin : float, optional
    Minimum value of the node color map.
node_color_vmax : float, optional
    Maximum value of the node color map.
node_color_center : float, optional
    Center value of the node color map.
n_cutoff : int
    The number of top reached pathways to plot.
node_alpha : float
    The alpha value of plotted nodes
node_color_title : str, optional
    Title of the node colorbar.
title : str, optional
    Title of the graph.
figsize : (float, float)
    Figure size.
```

Returns

None

**plot_pw_subgraph(
 G,
 pos,
 nodes,
 node_sizes,
 node_data,
 node_cmap,
 node_alpha,
 node_cbar_title,
 node_labels,
 edges,
 edge_widths,
 title,
 figsize=None,
 seed=42)** [Function]

**plot_rawk_sample_mtx(
 rawk_sample,
 save_pfx,
 base_figsize=2,
 vmin=None,
 vmax=None)** [Function]

pr_sort_key(x) [Function]

prs_to_ps(prs) [Function]

reset_xy_lim(ax, pos, ratio) [Function]

rawk.rawk

Rawk(sample, bg_sample) [Class]

Run Rawk on one sample vs background

Parameters

sample : RawkSample

An RawkSample instance to test for enrichment.

bg_sample : RawkSample

An RawkSample instance used as background.

Rawk.assert_graph_compatible(left, right) [Method]

Assert that two networkx graphs are compatible with Rawk

Rawk.assert_rs_compatible(left, right) [Method]

Assert that two RawkSample instances are compatible with Rawk

Rawk.set_node_emb(node_emb_method, pca_ndim=None, kmeans_n_clust=None) [Method]

Set node embeddings

Parameters

node_emb_method : str or dict

The method to set node embeddings, which can be one of the following options:

- 'fastrp': Compute 2D t-SNE from fastrp node embeddings.
- 'node2vec': Compute 2D t-SNE from node2vec node

embeddings.

This procedure may take several hours.

```

    - dict: Reuse pre-computed 2D embeddings of nodes stored
in a dict
    with the following format:
    {node_name1: (dim1_coord, dim2_coord), ...}. In this
option,
    pca_ndim and kmeans_n_clust should be None.
pca_ndim : int or None
    When node_emb_method in ['fastrp', 'node2vec'], the
number of PCA
    dimensions of the fastrp or node2vec embeddings to
compute 2D t-SNE
    and kmeans clusters. If None, default pca_ndim=6.
kmeans_n_clust : int or None
    When node_emb_method in ['fastrp', 'node2vec'], the
number of
    clusters parameter to run kmeans clustering. If None,
default
    pca_ndim=10.

Returns
-----
None

```

Rawk.test_num_steps(h_exponent=1, cmp_norm_fac=5000) [Method]

```

Enrichment test using the number of random walk steps

Parameters
-----
h_exponent : float
    The heuristic factor for exponentiating sample vs
background
    fold changes of the number of random walk steps. This
heuristic
    factor intends to distinguish pathways with relatively
small and
    large effect sizes.

cmp_norm_fac : float
    Normalization factor for the number of random walk steps.

```

Returns

A dictionary of testing results, with the following structure:

```
{
  sample_id: {
    pathway1: {'test_nes': test_nes, 'test_pval':
test_pval},
    ...
  }
}
```

**Rawk.test_property_values(rw_s_prop_cutoff=0.005,
pw_pvs_dict=None) [Method]**

Enrichment test using the node property values

Parameters

rw_s_prop_cutoff : float

The cutoff proportion of random walk steps used for
defining local
pathways.

pw_pvs_dict : dict or None

{pathway1: [property_value1, ...], ...} dict used for
extracting
pathway property values for testing. If None, use the
foreground
sample node properties.

Returns

A dictionary of testing results, with the following structure:

```
{
  sample_id: {
    pathway1: {'test_nes': test_nes, 'test_pval':
test_pval},
    ...
  }
}
```

RawkTest(fg_node_df, fg_df, bg_df, num_walks, test_id)

[Class]

Rawk test number of random walk steps

Parameters

fg_node_df : pd.DataFrame

A foreground dataframe of reactions. Following are the required columns:

'rxn' (reaction ID), 'pathway' (reaction pathway),

'property' (reaction property value),

'gene' (genes associated with the reaction). The 'gene' column lists

the genes associated with each reaction in a `;;;` joined list of gene

symbols.

fg_df : pd.DataFrame

A foreground (pathways, pathway_____reaction)

dataframe. (i, j) is the number of steps of all

random walks starting from j in pathway i.

bg_df : pd.DataFrame

A background (pathways, pathway_____reaction)

dataframe. (i, j) is the number of steps of all

random walks starting from j in pathway i.

num_walks : int

Number of random walks starting from each reaction.

test_id : str

ID for this test.

RawkTest.df_dict_to_df(jd) [Method]

Construct a dataframe from a dict

RawkTest.from_dict(dd) [Method]

Construct a RawkTest from a dict

RawkTest.get_a_test_ns_tbls(x, y, norm_fac, total_ns, h_exponent)
[Method]

RawkTest.get_pw_ns(xdf, ydf, pw) [Method]

RawkTest.get_spr(fg_df, bg_df) [Method]

RawkTest.load_json(file_path) [Method]

Read json files

Parameters

file_path : str
 Read file prefix.

Returns

A RawkTest constructed from the read data.

RawkTest.save_to_json(file_path) [Method]

Save data to a json file

Parameters

file_path : str
 Save json file path.

Returns

None

RawkTest.test_num_steps(h_exponent=1, cmp_norm_fac=5000) [Method]

```
Enrichment test using the number of random walk steps

Parameters
-----
h_exponent : float
    The heuristic factor for exponentiating sample vs
    background
    fold changes of the number of random walk steps. This
    heuristic
    factor intends to distinguish pathways with relatively
    small and
    large effect sizes.

cmp_norm_fac : float
    Normalization factor for the number of random walk steps.

Returns
-----
A dictionary of testing results, with the following structure:
{
    sample_id: {
        pathway1: {'test_nes': test_nes, 'test_pval':
test_pval},
        ...
    }
}
```

RawkTest.test_property_values(rw_s_prop_cutoff=0.005, pw_pvs_dict=None) [Method]

```
Enrichment test using the node property values

Parameters
-----
rw_s_prop_cutoff : float
    The cutoff proportion of random walk steps used for
    defininig local
```

```

    pathways.
pw_pvs_dict : dict or None
    {pathway1: [property_value1, ...], ...} dict used for
extracting
    pathway property values for testing. If None, use the
foreground
    sample node properties.

Returns
-----
A dictionary of testing results, with the following structure:
{
    sample_id: {
        pathway1: {'test_nes': test_nes, 'test_pval':
test_pval},
        ...
    }
}

```

RawkTest.test_res_dict_to_df(mnea_res_dict, m_sfx,
pw_subset=None) [Method]

```

Convert a test result dictionary to a 2-tuple of dataframes

Parameters
-----
mnea_res_dict : dict
    Rawk test result dictionary.

m_sfx : str
    Method suffix for dataframe columns.

pw_subset : set, or list, or None
    The subset of pathways to keep in the result dataframes.
    If is None, keep all pathways with not-NA results.

Returns
-----
A 2-tuple of (fdr_dataframe, enrichment_score_dataframe).

```

RawkTest.to_dict() [Method]

Convert data to a dict

rawk.rawk_sample

**RawkSample(
 node_df,
 edge_df,
 name='sample',
 uniform_property=False,
 workers=1,
 seed=17,
 node_emb_method='fastrp',
 n2v_walk_length=20,
 n2v_num_walks=8000,
 n2v_chunk_num_walks=1000,
 n2v_p=1.0,
 n2v_q=1.0,
 n2v_dimensions=50,
 n2v_window=10,
 n2v_epochs=1,
 random_walk_n_steps_method=None,
 keep_walks=False,
 verbose=False)** [Class]

Create a Rawk sample

Parameters

node_df : dataframe

A dataframe of reactions. Following are the required columns:

- 'rxn' (reaction ID), 'pathway' (reaction pathway),
- 'property' (reaction property value),
- 'gene' (genes associated with the reaction). The 'gene' column lists the genes associated with each reaction in a `;;;` joined list of gene symbols.

edge_df : dataframe

A dataframe of reactions edges. Following are the required columns:

- 'src' (source node reaction ID), 'dest' (destination node reaction ID),
- 'mn_weight' (metabolic network edge weight). The edges are undirected, with the `src <= dest` in alphabetical order.

name : str

The name of the sample.

uniform_property : bool

Whether the property values should be converted to the same value.

workers : int

The number of workers for running node2vec.

seed : int

Random number seed.

node_emb_method : str or dict

The method to set node embeddings, which can be one of the following options:

- 'fastrp': Compute 2D t-SNE from fastrp node embeddings.
- 'node2vec': Compute 2D t-SNE from node2vec node embeddings.

This procedure may take several hours.

- dict: Reuse pre-computed 2D embeddings of nodes stored in a dict with the following format:

```
{node_name1: (dim1_coord, dim2_coord), ...}.
```

n2v_walk_length : int

node2vec walk length parameter.

n2v_num_walks : int

node2vec number of walks parameter.

n2v_chunk_num_walks : int

```

    node2vec number of walks per chunk. Run node2vec in
chunks to reduce
    mem usage.
n2v_p : float
    node2vec p parameter.
n2v_q : float
    node2vec q parameter.
n2v_dimensions : int
    node2vec number of representation dimensions.
n2v_window : int
    node2vec window parameter.
n2v_epochs : int
    node2vec number of epochs parameter.
random_walk_n_steps_method : str
    Generate random walk number of steps with one of the
following methods:
    - 'simulation': Simulate random walks. Aggregate number
of steps.
    - 'calculation': Calculate number of steps using
transition probability
    matrix. NOTE: This method is only implemented for
n2v_p=1 and n2v_q=1.
    If node_emb_method='node2vec', the random walks still
need to be simulated.
    - None: When n2v_p=1 and n2v_q=1, use the calculation
method. Otherwise,
    use the simulation method.
verbose : bool
    Print runtime messages or not.

```

`RawkSample.most_frequent(x)` [Method]

`RawkSample.read_n2v_res(pfx)` [Method]

`RawkSample.run_kmeans(X, k, seed)` [Method]

`RawkSample.rxn_to_pw_rxn(df, index=True, columns=True)` [Method]

`RawkSample.save_n2v_res(pfx)` [Method]

`RawkSample.set_node_emb(node_emb_method, pca_ndim=None, kmeans_n_clust=None)` [Method]

Set node embeddings

Parameters

`node_emb_method` : str or dict

The method to set node embeddings, which can be one of the following options:

- 'fastrp': Compute 2D t-SNE from fastrp node embeddings.
- 'node2vec': Compute 2D t-SNE from node2vec node

embeddings.

This procedure may take several hours.

- dict: Reuse pre-computed 2D embeddings of nodes stored in a dict

with the following format:

{node_name1: (dim1_coord, dim2_coord), ...}. In this option,

`pca_ndim` and `kmeans_n_clust` should be None.

`pca_ndim` : int or None

When `node_emb_method` in ['fastrp', 'node2vec'], the number of PCA

dimensions of the fastrp or node2vec embeddings to compute 2D t-SNE

and kmeans clusters. If None, default `pca_ndim`=6.

`kmeans_n_clust` : int or None

When `node_emb_method` in ['fastrp', 'node2vec'], the number of

```
clusters parameter to run kmeans clustering. If None,  
default  
pca_ndim=10.
```

Returns

None

■ End of Document ■